Code

```
"""
General-Purpose Operations Orchestration Simulator
Eka Care–aligned, Governance-Safe

Engineering Principle:
> If a decision cannot be explained in one sentence to a tired nurse at 2 a.m.,
 it does not belong in this system.

This simulator:
- Does NOT automate decisions
- Does NOT optimize outcomes
- Does NOT use ML
- Does NOT rely on ABDM live data

It simulates, recommends, and logs.

FHIR / ABDM Compliance Note:
This system operates at the pre-interoperability orchestration layer.
All patient objects are losslessly mappable to FHIR R4 resources
(Patient, Encounter, Condition, Observation).
FHIR is treated as an interchange format, not a decision substrate.
ABDM data is not used for real-time logic, by design.
"""
import streamlit as st
import json
from dataclasses import dataclass, field
from typing import List, Dict, Optional
from enum import Enum
import random

#
================================================================================
======
# 3. CORE DOMAIN OBJECTS (DO NOT ADD FIELDS)
#
================================================================================
======

class PatientStatus(Enum):
    """Patient status - transitions are explicit and logged."""
    WAITING = "WAITING"
    ADMITTED = "ADMITTED"
    TRANSFERRED = "TRANSFERRED"
    DEFERRED = "DEFERRED"

@dataclass
class Patient:
    """
```

```
    Core patient object - fields are LOCKED per specification.
    DO NOT ADD FIELDS without updating the spec.

    NOTE: This class represents REALITY, not measurement.
    All derived metrics (wait times, admission times) must live in PatientMetrics.
    """
    id: int
    arrival_time: int  # seconds since simulation start
    chief_complaint: str
    age: int
    history: List[str]

    triage_stage_1: Optional[str] = None  # "RED" | "NOT_RED"
    triage_stage_2: Optional[str] = None  # "RED" | "YELLOW" | "BLUE"
    status: PatientStatus = PatientStatus.WAITING

@dataclass
class PatientMetrics:
    """
    Metrics derived from patient journey - separate from core domain.
    Populated ONLY from event logs, never written during logic execution.
    """
    patient_id: int
    admission_time: Optional[int] = None
    triage_2_start_time: Optional[int] = None
    wait_time_at_triage_2: int = 0

@dataclass
class Event:
    """
    Event log entry.
    Rule: If it's not logged, it did not happen.
    """
    timestamp: int
    type: str
    entity_id: Optional[int]
    details: Dict

@dataclass
class Room:
    """
    Hospital room with capacity constraints.
    Fixed types: Emergency | General OPD | Preventive Care | IPD
    """
    name: str
    room_type: str
    capacity_per_minute: int
    current_load: int = 0
```

```
@dataclass
class ChatBubble:
    """
    Social layer - effects only, never causes.
    Actors: Patient | Staff | Doctor | System
    Bubbles are deterministic (no LLMs, predefined phrases only).
    """
    timestamp: int
    actor: str
    message: str
    context: str

#
================================================================================
======
# 5. TRIAGE LOGIC (ARTIFACT 5)
#
================================================================================
======

# 5.1 Early Coarse Triage (Fail-Open)
# Conservative and dumb on purpose - no refinement here
RED_FLAG_KEYWORDS = [
    "chest pain", "unconscious", "severe bleeding", "stroke", "heart attack",
    "difficulty breathing", "choking", "severe trauma", "unresponsive",
    "seizure", "head injury", "severe burn", "collapse", "sweating",
    "radiating", "chest tightness", "shortness of breath", "neck swelling"
]

def early_triage(patient: Patient) -> str:
    """
    Early coarse triage - conservative and dumb on purpose.

    This is intentionally simple to err on the side of safety.
    Returns: "RED" or "NOT_RED"

    Rule: Conservative, fail-open. No refinement here.
    """
    complaint_lower = patient.chief_complaint.lower()

    for keyword in RED_FLAG_KEYWORDS:
        if keyword in complaint_lower:
            return "RED"

    return "NOT_RED"

def refined_triage(patient: Patient) -> str:
```

```
    """
    5.2 Late Refined Triage

    Uses age, history, and complaint for classification.
    Returns: "RED" | "YELLOW" | "BLUE"

    Runs only when:
    - Patient reaches front of queue, OR
    - After fixed wait threshold

    IRREVOCABLE RED RULE: Once triage_stage_2 == "RED", patient can never be
demoted.
    """
    # If already marked RED in stage 1, stays RED (irrevocable)
    if patient.triage_stage_1 == "RED":
        return "RED"

    complaint_lower = patient.chief_complaint.lower()

    # Age-based risk factors
    high_risk_age = patient.age > 65 or patient.age < 5

    # History-based risk factors
    high_risk_history = any(
        condition in " ".join(patient.history).lower()
        for condition in ["diabetes", "hypertension", "cardiac", "respiratory",
                          "hyperlipidemia", "smoking"]
    )

    # Moderate urgency keywords for YELLOW classification
    yellow_keywords = [
        "fracture", "infection", "high fever", "moderate pain", "severe pain",
        "vomiting", "dizziness", "rash", "wound", "abdominal pain",
        "loose motion", "swelling", "accident", "fell", "tooth came out"
    ]

    # Check for moderate urgency indicators
    is_yellow = any(keyword in complaint_lower for keyword in yellow_keywords)

    # Combine factors to determine triage level
    # High-risk patient with concerning symptoms gets YELLOW
    if high_risk_age and (high_risk_history or is_yellow):
        return "YELLOW"
    elif is_yellow:
        return "YELLOW"
    else:
        # Low acuity - routine care, checkups, minor complaints
        return "BLUE"
```

```
#
================================================================================
# 7. SEPARATION MONITOR (ARTIFACT 3)
#
================================================================================

def separation_monitor(queue: Dict[str, List[Patient]], current_time: int,
                       threshold: int = 3) -> bool:
    """
    Detect unsafe clustering of high-risk patients.

    Purpose: Prevent dangerous accumulation of RED patients waiting simultaneously.

    Rules:
    - Independent of queue ordering
    - Triggers escalation pathways
    - Does NOT act directly

    Returns: True if UNSAFE, False otherwise

    # TODO: Extend separation monitor to include
    # duration-based clustering (RED co-presence over time),
    # not just instantaneous count.
    """
    red_waiting = len(queue["RED"])

    if red_waiting >= threshold:
        return True

    return False

#
================================================================================
# 8. GOVERNANCE & ESCALATION (ARTIFACT 1)
#
================================================================================

def governance_check(queue: Dict[str, List[Patient]], rooms: List[Room],
                     current_time: int) -> List[str]:
    """
    8.1 Governance Gate

    Returns recommendations only - NO automatic execution.
```

```
    All recommendations are surfaced to UI for human decision-making.

    This preserves human authority while providing system intelligence.
    """
    recommendations = []

    # Check separation safety using the separation monitor
    if separation_monitor(queue, current_time):
        recommendations.extend([
            "SUGGEST_EXTERNAL_REFERRAL",
            "SUGGEST_ROOM_MORPH",
            "SUGGEST_REAPPOINTMENT"
        ])

    # Check overall queue pressure
    total_waiting = sum(len(q) for q in queue.values())
    if total_waiting > 15:  # Threshold for general pressure
        recommendations.append("SUGGEST_CAPACITY_INCREASE")

    # Check emergency room utilization
    emergency_rooms = [r for r in rooms if r.room_type == "Emergency"]
    if emergency_rooms:
        total_capacity = sum(r.capacity_per_minute for r in emergency_rooms)
        total_load = sum(r.current_load for r in emergency_rooms)
        if total_capacity > 0:
            utilization = total_load / total_capacity
            if utilization > 0.85:
                recommendations.append("SUGGEST_EMERGENCY_BACKUP")

    return recommendations

#
================================================================================
======
# 12. CHAT BUBBLES (SOCIAL LAYER)
#
================================================================================
======

# Predefined phrases - deterministic, no LLMs
CHAT_PHRASES = {
    "patient_complaint_reorder": [
        "Why did that person go ahead of me?",
        "I've been waiting for so long...",
        "Is there a problem with my case?",
        "When will it be my turn?"
    ],
    "patient_complaint_wait": [
```

```
        "How much longer do I have to wait?",
        "I arrived before them, why are they going first?",
        "This waiting time is too long."
    ],
    "staff_explanation": [
        "Your case is being carefully reviewed.",
        "We're managing multiple urgent cases right now.",
        "A doctor will see you as soon as possible.",
        "We prioritize based on medical urgency, not arrival time."
    ],
    "system_escalation": [
        "Alert: High volume of urgent cases detected.",
        "Recommendation: Consider external referral options.",
        "System suggests capacity review.",
        "Warning: Emergency department at high utilization."
    ],
    "staff_reassurance_deferral": [
        "We can schedule you for tomorrow when we have more capacity.",
        "Your condition can be safely managed with a follow-up appointment.",
        "We'll ensure you get the care you need at the right time.",
        "Let's book you a proper appointment slot."
    ],
    "doctor_triage": [
        "Reviewing patient urgency levels.",
        "Assessing immediate medical needs.",
        "Prioritizing critical cases."
    ]
}

def generate_chat_bubble(event: Event, patient: Optional[Patient] = None) ->
Optional[ChatBubble]:
    """
    Generate chat bubbles based on events.

    Rule: Bubbles are effects only, never causes.
    They reflect system state but do not influence logic.
    """
    if event.type == "QUEUE_REORDERED" and patient:
        return ChatBubble(
            timestamp=event.timestamp,
            actor="Patient",
            message=random.choice(CHAT_PHRASES["patient_complaint_reorder"]),
            context="Queue reordered due to triage"
        )

    elif event.type == "ESCALATION_SUGGESTED":
        return ChatBubble(
            timestamp=event.timestamp,
```

```
            actor="System",
            message=random.choice(CHAT_PHRASES["system_escalation"]),
            context="Governance escalation triggered"
        )

    elif event.type == "PATIENT_DEFERRED" and patient:
        return ChatBubble(
            timestamp=event.timestamp,
            actor="Staff",
            message=random.choice(CHAT_PHRASES["staff_reassurance_deferral"]),
            context=f"Patient {patient.id} deferred"
        )

    elif event.type == "TRIAGE_STAGE_2_ASSIGNED" and patient:
        # Occasionally generate patient or staff bubbles during triage
        if random.random() < 0.3:  # 30% chance of chat activity
            if patient.triage_stage_2 == "RED":
                return ChatBubble(
                    timestamp=event.timestamp,
                    actor="Doctor",
                    message=random.choice(CHAT_PHRASES["doctor_triage"]),
                    context=f"Triaging patient {patient.id}"
                )
            elif random.random() < 0.5:
                return ChatBubble(
                    timestamp=event.timestamp,
                    actor="Patient",
                    message=random.choice(CHAT_PHRASES["patient_complaint_wait"]),
                    context=f"Patient {patient.id} waiting"
                )

    elif event.type == "PATIENT_ADMITTED" and patient and random.random() < 0.2:
        return ChatBubble(
            timestamp=event.timestamp,
            actor="Staff",
            message=random.choice(CHAT_PHRASES["staff_explanation"]),
            context=f"Patient {patient.id} admitted"
        )

    return None

#
==========================================================================
======
# 10. SIMULATION ENGINE
#
==========================================================================
======
```

```
class SimulationEngine:
    """
    Core simulation engine implementing the deterministic orchestration system.

    4.1 Agent Loop (IMMUTABLE):
    Every simulation tick executes:
    PERCEIVE → CLASSIFY → ORDER → CHECK → SURFACE → LOG

    No step may be skipped or reordered.
    """

    def __init__(self, profile: str = "Balanced", use_test_data: bool = False, custom_dataset:
Optional[List[Dict]] = None):
        self.profile = profile
        self.use_test_data = use_test_data
        self.custom_dataset = custom_dataset
        self.current_time = 0
        self.duration = 300  # 5 minutes in seconds
        self.arrival_interval = 5  # New patient every 5 seconds

        # Core state
        self.patients: List[Patient] = []
        self.queue: Dict[str, List[Patient]] = {
            "RED": [],
            "YELLOW": [],
            "BLUE": []
        }

        # Logging and social layer
        self.events: List[Event] = []
        self.chat_bubbles: List[ChatBubble] = []

        # Infrastructure
        self.rooms = self._initialize_rooms(profile)
        self.patient_id_counter = 0

        # Tracking for post-run evaluation
        self.admitted_patients: List[Patient] = []
        self.deferred_patients: List[Patient] = []
        self.transferred_patients: List[Patient] = []

        # Metrics (separate from core domain)
        self.patient_metrics: Dict[int, PatientMetrics] = {}

        # Load test data if requested
        if use_test_data:
            if custom_dataset:
```

```
            self.test_dataset = custom_dataset
        else:
            self.test_dataset = self._load_test_dataset()
        self.test_data_index = 0

    def _load_test_dataset(self) -> List[Dict]:
        """Load the provided test dataset."""
        return [
            {"id": 1, "arrival_time": 0, "chief_complaint": "severe chest pain radiating to left arm",
"age": 62, "history": ["hypertension", "hyperlipidemia"]},
            {"id": 2, "arrival_time": 5, "chief_complaint": "routine blood pressure check", "age": 45,
"history": ["hypertension"]},
            {"id": 3, "arrival_time": 10, "chief_complaint": "tooth pain cannot sleep at night", "age":
34, "history": []},
            {"id": 4, "arrival_time": 15, "chief_complaint": "bleeding gums wants cleaning", "age":
29, "history": []},
            {"id": 5, "arrival_time": 20, "chief_complaint": "difficulty breathing and chest
tightness", "age": 58, "history": ["diabetes"]},
            {"id": 6, "arrival_time": 25, "chief_complaint": "follow up visit for diabetes review",
"age": 51, "history": ["diabetes"]},
            {"id": 7, "arrival_time": 30, "chief_complaint": "fever for 2 days no other symptoms",
"age": 22, "history": []},
            {"id": 8, "arrival_time": 35, "chief_complaint": "fell down bike accident tooth came
out", "age": 18, "history": []},
            {"id": 9, "arrival_time": 40, "chief_complaint": "routine dental scaling appointment",
"age": 41, "history": []},
            {"id": 10, "arrival_time": 45, "chief_complaint": "sudden severe abdominal pain right
side", "age": 27, "history": []},
            {"id": 11, "arrival_time": 50, "chief_complaint": "headache wants pain medicine",
"age": 36, "history": []},
            {"id": 12, "arrival_time": 55, "chief_complaint": "shortness of breath worsening", "age":
70, "history": ["cardiac disease"]},
            {"id": 13, "arrival_time": 60, "chief_complaint": "gum swelling mild pain", "age": 33,
"history": []},
            {"id": 14, "arrival_time": 65, "chief_complaint": "annual health checkup", "age": 39,
"history": []},
            {"id": 15, "arrival_time": 70, "chief_complaint": "vomiting and loose motion", "age": 26,
"history": []},
            {"id": 16, "arrival_time": 75, "chief_complaint": "follow up for blood test reports", "age":
48, "history": ["hypertension"]},
            {"id": 17, "arrival_time": 80, "chief_complaint": "jaw swelling difficulty opening mouth",
"age": 44, "history": []},
            {"id": 18, "arrival_time": 85, "chief_complaint": "routine pregnancy counseling", "age":
30, "history": []},
            {"id": 19, "arrival_time": 90, "chief_complaint": "chest pain since morning sweating",
"age": 55, "history": ["smoking"]},
            {"id": 20, "arrival_time": 95, "chief_complaint": "dental sensitivity to cold", "age": 21,
"history": []},
```

```
        {"id": 21, "arrival_time": 100, "chief_complaint": "difficulty breathing and neck
swelling", "age": 46, "history": []},
        {"id": 22, "arrival_time": 105, "chief_complaint": "follow up for cholesterol", "age": 52,
"history": ["hyperlipidemia"]},
        {"id": 23, "arrival_time": 110, "chief_complaint": "mild cough and cold", "age": 19,
"history": []},
        {"id": 24, "arrival_time": 115, "chief_complaint": "routine dental check", "age": 37,
"history": []},
        {"id": 25, "arrival_time": 120, "chief_complaint": "severe chest pain collapse", "age":
63, "history": ["hypertension", "diabetes"]},
    ]

  def _initialize_rooms(self, profile: str) -> List[Room]:
    """
    9.1 Rooms (Fixed Types)
    Initialize rooms based on hospital profile.

    Each room has: capacity_per_minute, current_load
    """
    if profile == "Govt":
      # Government hospital - limited resources
      return [
        Room("Emergency 1", "Emergency", capacity_per_minute=1),
        Room("Emergency 2", "Emergency", capacity_per_minute=1),
        Room("OPD 1", "General OPD", capacity_per_minute=2),
        Room("OPD 2", "General OPD", capacity_per_minute=2),
      ]
    elif profile == "Private":
      # Private hospital - more resources
      return [
        Room("Emergency 1", "Emergency", capacity_per_minute=2),
        Room("Emergency 2", "Emergency", capacity_per_minute=2),
        Room("OPD 1", "General OPD", capacity_per_minute=3),
        Room("OPD 2", "General OPD", capacity_per_minute=3),
        Room("Preventive Care", "Preventive Care", capacity_per_minute=2),
      ]
    else:  # Balanced
      # Balanced hospital - moderate resources
      return [
        Room("Emergency 1", "Emergency", capacity_per_minute=1),
        Room("Emergency 2", "Emergency", capacity_per_minute=2),
        Room("OPD 1", "General OPD", capacity_per_minute=2),
        Room("OPD 2", "General OPD", capacity_per_minute=3),
        Room("Preventive Care", "Preventive Care", capacity_per_minute=1),
      ]

  def _generate_patient_from_test_data(self) -> Optional[Patient]:
    """Generate patient from test dataset if available."""
```

```
    if self.test_data_index >= len(self.test_dataset):
        return None

    # Find next patient whose arrival time matches current time
    while self.test_data_index < len(self.test_dataset):
        data = self.test_dataset[self.test_data_index]
        if data["arrival_time"] == self.current_time:
            self.test_data_index += 1
            patient = Patient(
                id=data["id"],
                arrival_time=data["arrival_time"],
                chief_complaint=data["chief_complaint"],
                age=data["age"],
                history=data["history"]
            )
            # Initialize metrics for this patient
            self.patient_metrics[patient.id] = PatientMetrics(patient_id=patient.id)
            return patient
        elif data["arrival_time"] > self.current_time:
            # Not time yet for this patient
            return None
        else:
            # Missed this patient (shouldn't happen)
            self.test_data_index += 1

    return None

def _generate_patient_random(self) -> Patient:
    """Generate a random patient with realistic complaints and demographics."""
    self.patient_id_counter += 1

    # Realistic complaint distribution
    complaints = [
        ("chest pain", 0.05),
        ("difficulty breathing", 0.05),
        ("severe bleeding", 0.03),
        ("fracture", 0.10),
        ("high fever", 0.15),
        ("moderate pain", 0.20),
        ("cough and cold", 0.15),
        ("headache", 0.12),
        ("skin rash", 0.10),
        ("routine checkup", 0.05)
    ]

    complaint = random.choices(
        [c[0] for c in complaints],
        weights=[c[1] for c in complaints]
```

```
    )[0]

    # Age distribution
    age = random.choices(
        [random.randint(0, 10), random.randint(18, 45),
         random.randint(45, 65), random.randint(65, 90)],
        weights=[0.15, 0.40, 0.30, 0.15]
    )[0]

    # Medical history based on age
    history = []
    if age > 50:
        if random.random() < 0.4:
            history.append("hypertension")
        if random.random() < 0.3:
            history.append("diabetes")

    patient = Patient(
        id=self.patient_id_counter,
        arrival_time=self.current_time,
        chief_complaint=complaint,
        age=age,
        history=history
    )

    # Initialize metrics for this patient
    self.patient_metrics[patient.id] = PatientMetrics(patient_id=patient.id)

    return patient

def _log_event(self, event_type: str, entity_id: Optional[int] = None,
               details: Dict = None):
    """
    11. EVENT LOGGING (MANDATORY)

    Rule: If it's not logged, it did not happen.
    Every meaningful action produces an event.
    """
    event = Event(
        timestamp=self.current_time,
        type=event_type,
        entity_id=entity_id,
        details=details or {}
    )
    self.events.append(event)

    # Generate chat bubble if applicable (social layer is effect, not cause)
    patient = None
```

```
    if entity_id:
        patient = next((p for p in self.patients if p.id == entity_id), None)

    bubble = generate_chat_bubble(event, patient)
    if bubble:
        self.chat_bubbles.append(bubble)

def _perceive(self):
    """
    STEP 1: PERCEIVE
    Check for new patient arrivals.
    """
    if self.use_test_data:
        # Use test dataset
        patient = self._generate_patient_from_test_data()
        if patient:
            self.patients.append(patient)
            self._log_event("PATIENT_ARRIVED", patient.id, {
                "complaint": patient.chief_complaint,
                "age": patient.age,
                "history": patient.history
            })

            # 5.1 Immediate early triage (fail-open, conservative)
            patient.triage_stage_1 = early_triage(patient)
            self._log_event("TRIAGE_STAGE_1_ASSIGNED", patient.id, {
                "triage": patient.triage_stage_1,
                "reason": "Early coarse triage on arrival"
            })
    else:
        # Generate random patients
        if self.current_time % self.arrival_interval == 0 and self.current_time < self.duration:
            patient = self._generate_patient_random()
            self.patients.append(patient)
            self._log_event("PATIENT_ARRIVED", patient.id, {
                "complaint": patient.chief_complaint,
                "age": patient.age
            })

            # Immediate early triage
            patient.triage_stage_1 = early_triage(patient)
            self._log_event("TRIAGE_STAGE_1_ASSIGNED", patient.id, {
                "triage": patient.triage_stage_1
            })

def _classify(self):
    """
    STEP 2: CLASSIFY
```

```
    Perform refined triage for eligible patients.

    5.2 Late Refined Triage runs only when:
    - Patient reaches front of queue, OR
    - After fixed wait threshold
    """
    wait_threshold = 60  # seconds before forcing refined triage

    for patient in self.patients:
        if patient.status != PatientStatus.WAITING:
            continue

        if patient.triage_stage_2 is None:
            # Check if patient is at front of any queue
            at_front = False
            for band, queue in self.queue.items():
                if queue and queue[0].id == patient.id:
                    at_front = True
                    break

            # Or if patient has waited long enough
            waited_long = (self.current_time - patient.arrival_time) > wait_threshold

            # Perform refined triage if conditions met
            if at_front or waited_long:
                patient.triage_stage_2 = refined_triage(patient)
                wait_time = self.current_time - patient.arrival_time

                # Update metrics (derived from events)
                if patient.id in self.patient_metrics:
                    self.patient_metrics[patient.id].triage_2_start_time = self.current_time
                    self.patient_metrics[patient.id].wait_time_at_triage_2 = wait_time

                self._log_event("TRIAGE_STAGE_2_ASSIGNED", patient.id, {
                    "triage": patient.triage_stage_2,
                    "wait_time": wait_time,
                    "reason": "At front of queue" if at_front else "Wait threshold exceeded"
                })

def _order(self):
    """
    STEP 3: ORDER
    Organize queue by triage bands.

    6.1 Queue Structure Rules:
    - FIFO within each band
    - RED always ahead of others
    - NO global sorting
```

```
    - NO priority scoring

    6.2 Irrevocable RED Rule:
    Once triage_stage_2 == "RED", patient can never be demoted.
    """
    # Store previous queue state to detect reordering
    prev_queue_state = {
        band: [p.id for p in patients]
        for band, patients in self.queue.items()
    }

    # Clear and rebuild queues
    self.queue = {"RED": [], "YELLOW": [], "BLUE": []}

    # Add patients to appropriate queues (preserving arrival order = FIFO)
    for patient in self.patients:
        if patient.status == PatientStatus.WAITING and patient.triage_stage_2:
            # IRREVOCABLE RED RULE enforced here
            self.queue[patient.triage_stage_2].append(patient)

    # Check if queue order changed (for chat bubble generation)
    new_queue_state = {
        band: [p.id for p in patients]
        for band, patients in self.queue.items()
    }

    if prev_queue_state != new_queue_state:
        self._log_event("QUEUE_REORDERED", None, {
            "previous_state": prev_queue_state,
            "new_state": new_queue_state
        })

def _check_governance(self) -> List[str]:
    """
    STEP 4: CHECK (governance)

    8.1 Governance Gate returns recommendations only.
    NO automatic execution - preserves human authority.
    """
    return governance_check(self.queue, self.rooms, self.current_time)

def _surface_recommendations(self, recommendations: List[str]):
    """
    STEP 5: SURFACE

    Log and display recommendations (never execute automatically).
    All recommendations require human approval.
    """
```

```
        if recommendations:
            self._log_event("ESCALATION_SUGGESTED", None, {
                "recommendations": recommendations,
                "reason": "Governance check triggered"
            })

    def _admit_patients(self):
        """
        10.2 Admission Logic

        NOTE:
        Admission represents physical room availability and elapsed consult time.
        This is NOT an AI decision and does not override human authority.

        At each tick:
        - Rooms admit patients from queue based on capacity
        - RED patients always considered first
        - Admissions logged
        """
        # Priority order: RED always first
        priority_order = ["RED", "YELLOW", "BLUE"]

        for band in priority_order:
            for patient in list(self.queue[band]):  # Copy to avoid modification during iteration
                admitted = False

                # RED patients go to Emergency rooms
                if band == "RED":
                    for room in self.rooms:
                        if room.room_type == "Emergency" and room.current_load <
room.capacity_per_minute:
                            patient.status = PatientStatus.ADMITTED
                            room.current_load += 1
                            self.admitted_patients.append(patient)
                            self.queue[band].remove(patient)

                            # Update metrics (derived from events)
                            if patient.id in self.patient_metrics:
                                self.patient_metrics[patient.id].admission_time = self.current_time

                            wait_time = self.current_time - patient.arrival_time
                            self._log_event("PATIENT_ADMITTED", patient.id, {
                                "room": room.name,
                                "wait_time": wait_time,
                                "triage": patient.triage_stage_2
                            })
                            admitted = True
                            break
```

```
            # Non-emergency cases go to OPD or Preventive Care
            else:
                for room in self.rooms:
                    if room.room_type in ["General OPD", "Preventive Care"] and \
                       room.current_load < room.capacity_per_minute:
                        patient.status = PatientStatus.ADMITTED
                        room.current_load += 1
                        self.admitted_patients.append(patient)
                        self.queue[band].remove(patient)

                        # Update metrics (derived from events)
                        if patient.id in self.patient_metrics:
                            self.patient_metrics[patient.id].admission_time = self.current_time

                        wait_time = self.current_time - patient.arrival_time
                        self._log_event("PATIENT_ADMITTED", patient.id, {
                            "room": room.name,
                            "wait_time": wait_time,
                            "triage": patient.triage_stage_2
                        })
                        admitted = True
                        break

    # Simulated consult completion:
    # every 30 seconds, one patient exits the room,
    # approximating average consult duration.
    if self.current_time % 30 == 0:
        for room in self.rooms:
            room.current_load = max(0, room.current_load - 1)

def tick(self):
    """
    Execute one simulation tick.

    4.1 Agent Loop (IMMUTABLE):
    PERCEIVE → CLASSIFY → ORDER → CHECK → SURFACE → LOG

    No step may be skipped or reordered.
    """
    self.current_time += 1

    # 1. PERCEIVE - check for new arrivals
    self._perceive()

    # 2. CLASSIFY - perform refined triage
    self._classify()
```

```
        # 3. ORDER - organize queues by triage bands
        self._order()

        # 4. CHECK - governance gate for recommendations
        recommendations = self._check_governance()

        # 5. SURFACE - display recommendations to humans
        self._surface_recommendations(recommendations)

        # 6. Process admissions (part of orchestration)
        self._admit_patients()

        # Note: LOG happens throughout via _log_event()

    def run_full_simulation(self):
        """Run the complete simulation and generate report."""
        while self.current_time < self.duration:
            self.tick()

        return self.generate_report()

    def generate_report(self) -> Dict:
        """
        Generate post-run evaluation report.

        Three scores: IES, PSS, SSS
        All computed AFTER simulation, never shown live, never feed logic.
        """
        return {
            "ies": self._calculate_ies(),
            "pss": self._calculate_pss(),
            "sss": self._calculate_sss(),
            "summary": self._generate_summary()
        }

    def _calculate_ies(self) -> Dict[str, float]:
        """
        13. INSTITUTIONAL EFFICACY SCORE (IES) – POST-RUN

        Five pillars, each scored 0–100:
        1. Safety Preservation
        2. Dignity & Fairness
        3. Flow Stability
        4. Capacity Adaptation
        5. Human Authority Integrity

        IES is computed after simulation, never shown live, never feeds logic.
        """
```

```
        # 1. Safety Preservation - did we handle RED patients well?
        red_patients = [p for p in self.patients if p.triage_stage_2 == "RED"]
        red_admitted = [p for p in red_patients if p.status == PatientStatus.ADMITTED]
        safety_score = (len(red_admitted) / len(red_patients) * 100) if red_patients else 100

        # 2. Dignity & Fairness - minimize complaints from queue reordering
        queue_reorders = sum(1 for e in self.events if e.type == "QUEUE_REORDERED")
        dignity_score = max(0, 100 - (queue_reorders * 3))

        # 3. Flow Stability - manage wait times effectively
        if self.admitted_patients:
            total_wait = sum(
                self.patient_metrics[p.id].admission_time - p.arrival_time
                for p in self.admitted_patients
                if p.id in self.patient_metrics and self.patient_metrics[p.id].admission_time
            )
            avg_wait = total_wait / len(self.admitted_patients)
            # Penalize long average waits (over 2 minutes = 120 seconds)
            flow_score = max(0, 100 - (avg_wait / 2))
        else:
            flow_score = 0

        # 4. Capacity Adaptation - did we appropriately escalate?
        escalations = sum(1 for e in self.events if e.type == "ESCALATION_SUGGESTED")
        # Reward appropriate escalations (but not too many)
        adaptation_score = min(100, escalations * 15)
        adaptation_score = max(0, adaptation_score - (escalations - 5) * 10) if escalations > 5
else adaptation_score

        # 5. Human Authority Integrity - no autonomous decisions made
        # This is always 100 in this system by design
        authority_score = 100

        overall = (safety_score + dignity_score + flow_score + adaptation_score +
authority_score) / 5

        return {
            "Safety Preservation": round(safety_score, 2),
            "Dignity & Fairness": round(dignity_score, 2),
            "Flow Stability": round(flow_score, 2),
            "Capacity Adaptation": round(adaptation_score, 2),
            "Human Authority Integrity": round(authority_score, 2),
            "Overall": round(overall, 2)
        }

    def _calculate_pss(self) -> float:
        """
        14. PATIENT SATISFACTION SCORE (PSS) – FORMULA
```

Exact frozen formula from spec:
PSS = 0.30 * SilentAcceptance +
    0.25 * ExplanationSuccess +
    0.20 * DeferralTrust +
    0.15 * ComplaintPenalty +
    0.10 * ExitPenalty

Each component normalized to [0,1] first.
Scaled to 0–100.

Interpretation: Measures fairness & predictability, NOT happiness, NOT wait time.

Implementation Pattern:
SAC = N_reorders_silent / max(N_reorders, 1)
ESC = N_ack / max(N_confusion + 1, 1)
DTC = N_deferred_kept / max(N_deferred + 1, 1)
CPC = 1 - (N_repeat_confusion / max(N_patients, 1))
EPC = 1 - (N_early_exit / max(N_patients, 1))
"""
```
total_patients = len(self.patients)
if total_patients == 0:
    return 100.0

# Count key events
queue_reorders = sum(1 for e in self.events if e.type == "QUEUE_REORDERED")
patient_complaints = sum(1 for b in self.chat_bubbles if b.actor == "Patient")
staff_explanations = sum(1 for b in self.chat_bubbles if b.actor == "Staff")
deferral_count = len(self.deferred_patients)
still_waiting = sum(len(q) for q in self.queue.values())

# 1. Silent Acceptance Component (SAC)
# Proportion of queue reorderings that didn't generate complaints
if queue_reorders > 0:
    complaints_about_reorder = sum(1 for b in self.chat_bubbles
                                   if b.actor == "Patient" and "ahead" in b.message.lower())
    silent_reorders = max(0, queue_reorders - complaints_about_reorder)
    SAC = silent_reorders / queue_reorders
else:
    SAC = 1.0  # No reorders = perfect acceptance

# 2. Explanation Success Component (ESC)
# How well staff responded to patient concerns
if patient_complaints > 0:
    ESC = min(1.0, staff_explanations / patient_complaints)
else:
    ESC = 1.0  # No complaints = perfect
```

```
    # 3. Deferral Trust Component (DTC)
    # Patients who accepted deferral recommendations
    if deferral_count > 0:
        # In our model, deferred patients are those explicitly marked
        # We approximate "kept" deferrals as those who were deferred
        DTC = 1.0 - (deferral_count / max(1, total_patients))
    else:
        DTC = 1.0  # No deferrals needed = perfect

    # 4. Complaint Penalty Component (CPC)
    # Inverse of repeat confusion rate
    CPC = max(0.0, 1.0 - (patient_complaints / total_patients))

    # 5. Exit Penalty Component (EPC)
    # Patients who left without being seen
    EPC = max(0.0, 1.0 - (still_waiting / total_patients))

    # Clamp all components to [0, 1]
    SAC = max(0.0, min(1.0, SAC))
    ESC = max(0.0, min(1.0, ESC))
    DTC = max(0.0, min(1.0, DTC))
    CPC = max(0.0, min(1.0, CPC))
    EPC = max(0.0, min(1.0, EPC))

    # Apply exact formula from spec
    PSS_raw = (
        0.30 * SAC +
        0.25 * ESC +
        0.20 * DTC +
        0.15 * CPC +
        0.10 * EPC
    )

    # Scale to 0-100
    PSS = PSS_raw * 100

    return round(PSS, 2)

def _calculate_sss(self) -> float:
    """
    15. STAFF SATISFACTION SCORE (SSS) – FORMULA

    Based on:
    - Explanation load
    - De-escalation frequency
    - Override clustering
    - Alert density
```

```
        Higher cognitive load → lower SSS
        Computed post-run only.
        """
        # Explanation load - how many times staff had to explain things
        staff_messages = sum(1 for b in self.chat_bubbles if b.actor == "Staff")
        explanation_load_score = max(0, 100 - (staff_messages * 4))

        # Alert density - how many escalation alerts
        alerts = sum(1 for e in self.events if e.type == "ESCALATION_SUGGESTED")
        alert_density_score = max(0, 100 - (alerts * 8))

        # De-escalation frequency - patient complaints requiring attention
        patient_concerns = sum(1 for b in self.chat_bubbles if b.actor == "Patient")
        deescalation_score = max(0, 100 - (patient_concerns * 3))

        # Override clustering - triage changes (indicates decision difficulty)
        triage_assignments = sum(1 for e in self.events if e.type in
["TRIAGE_STAGE_1_ASSIGNED", "TRIAGE_STAGE_2_ASSIGNED"])
        override_score = max(0, 100 - (triage_assignments / max(1, len(self.patients)) * 20))

        # Average all factors
        sss = (explanation_load_score + alert_density_score + deescalation_score +
override_score) / 4

        return round(sss, 2)

    def _generate_summary(self) -> Dict:
        """Generate human-readable summary statistics."""
        # Calculate average wait time from metrics
        avg_wait = 0.0
        if self.admitted_patients:
            total_wait = sum(
                self.patient_metrics[p.id].admission_time - p.arrival_time
                for p in self.admitted_patients
                if p.id in self.patient_metrics and self.patient_metrics[p.id].admission_time
            )
            avg_wait = total_wait / len(self.admitted_patients)

        return {
            "total_patients": len(self.patients),
            "admitted": len(self.admitted_patients),
            "waiting": sum(len(q) for q in self.queue.values()),
            "deferred": len(self.deferred_patients),
            "transferred": len(self.transferred_patients),
            "red_patients": sum(1 for p in self.patients if p.triage_stage_2 == "RED"),
            "yellow_patients": sum(1 for p in self.patients if p.triage_stage_2 == "YELLOW"),
            "blue_patients": sum(1 for p in self.patients if p.triage_stage_2 == "BLUE"),
            "total_events": len(self.events),
```

```
            "escalations": sum(1 for e in self.events if e.type == "ESCALATION_SUGGESTED"),
            "avg_wait_time": avg_wait
        }

# ============================================================================
# 16. STREAMLIT UI REQUIREMENTS
# ============================================================================

def main():
    """
    Streamlit UI with three main sections:
    1. Simulation Controls
    2. Live View
    3. Post-Run Report
    """
    st.set_page_config(
        page_title="Hospital Operations Orchestration Simulator",
        page_icon="🏥",
        layout="wide"
    )

    # Main title
    st.title("🏥 General-Purpose Operations Orchestration Simulator")
    st.markdown("**Eka Care–aligned, Governance-Safe**")
    st.markdown("---")

    # Sidebar - Simulation Controls
    with st.sidebar:
        st.header("⚙️ Simulation Controls")

        # Profile selection
        profile = st.selectbox(
            "Hospital Profile",
            ["Govt", "Private", "Balanced"],
            index=2,
            help="Different profiles have different room capacities and resources"
        )

        # Data source selection
        data_source = st.radio(
            "Patient Data Source",
            ["Built-in Test Dataset", "Random Generation", "Upload Custom CSV"],
            help="Choose how to generate patient data"
        )
```

```
        custom_dataset = None
        use_test_data = False

        if data_source == "Built-in Test Dataset":
            use_test_data = True
            st.info("📊 Using 25-patient test dataset for deterministic results")

        elif data_source == "Upload Custom CSV":
            use_test_data = True
            st.markdown("**📁 Upload Patient CSV File**")

            # Sample CSV template download
            sample_csv = """id,arrival_time,chief_complaint,age,history
1,0,severe chest pain radiating to left arm,62,"hypertension,hyperlipidemia"
2,5,routine blood pressure check,45,hypertension
3,10,tooth pain cannot sleep at night,34,
4,15,bleeding gums wants cleaning,29,
5,20,difficulty breathing and chest tightness,58,diabetes
6,25,follow up visit for diabetes review,51,diabetes
7,30,fever for 2 days no other symptoms,22,
8,35,fell down bike accident tooth came out,18,
9,40,routine dental scaling appointment,41,
10,45,sudden severe abdominal pain right side,27,"""

            st.download_button(
                label="📥 Download Sample CSV Template",
                data=sample_csv,
                file_name="sample_patient_dataset.csv",
                mime="text/csv",
                help="Download a sample CSV file to see the required format"
            )

            uploaded_file = st.file_uploader(
                "Choose CSV file",
                type=['csv'],
                help="CSV must have columns: id, arrival_time, chief_complaint, age, history"
            )

            if uploaded_file is not None:
                try:
                    import pandas as pd
                    import io

                    # Read CSV
                    df = pd.read_csv(uploaded_file)

                    # Validate required columns
```

```
            required_columns = ['id', 'arrival_time', 'chief_complaint', 'age', 'history']
            missing_columns = [col for col in required_columns if col not in df.columns]

            if missing_columns:
                st.error(f"❌ Missing required columns: {', '.join(missing_columns)}")
                st.info("Required columns: id, arrival_time, chief_complaint, age, history")
            else:
                # Convert dataframe to list of dicts
                custom_dataset = []
                for _, row in df.iterrows():
                    # Parse history (can be string or list)
                    history = row['history']
                    if isinstance(history, str):
                        if history.strip() == '' or history.lower() == 'nan':
                            history = []
                        else:
                            # Try to parse as list or split by comma
                            history = [h.strip() for h in history.split(',') if h.strip()]
                    elif pd.isna(history):
                        history = []

                    custom_dataset.append({
                        'id': int(row['id']),
                        'arrival_time': int(row['arrival_time']),
                        'chief_complaint': str(row['chief_complaint']),
                        'age': int(row['age']),
                        'history': history
                    })

                st.success(f"✅ Loaded {len(custom_dataset)} patients from CSV")

                # Show preview
                with st.expander("Preview uploaded data"):
                    st.dataframe(df.head(10))

        except Exception as e:
            st.error(f"❌ Error reading CSV: {str(e)}")
            st.info("Please ensure your CSV is properly formatted")
    else:
        st.warning("⚠️ Please upload a CSV file to use custom data")
        st.markdown("""
        **CSV Format Example:**
        ```
        id,arrival_time,chief_complaint,age,history
        1,0,severe chest pain,62,"hypertension,diabetes"
        2,5,routine checkup,45,
        3,10,headache,34,
        ```
```

```
            """)

        else:  # Random Generation
            st.info("🎲 Generating random patients during simulation")

        st.markdown("---")

        # Start button
        if st.button("🚀 Start Simulation", type="primary", use_container_width=True):
            # Check if custom CSV is required but not uploaded
            if data_source == "Upload Custom CSV" and custom_dataset is None:
                st.error("Please upload a CSV file first!")
            else:
                st.session_state.simulation_running = True
                st.session_state.simulation = SimulationEngine(profile, use_test_data,
custom_dataset)
                st.session_state.report = None
                st.session_state.completed = False

        # Reset button
        if st.button("🔄 Reset", use_container_width=True):
            st.session_state.clear()
            st.rerun()

        st.markdown("---")

        # About section
        st.markdown("### 📖 About This System")
        st.markdown("""
        This simulator demonstrates deterministic queue management while preserving safety,
human authority, and ethical governance.

        **Core Principles:**
        - Safety first
        - Human authority preserved
        - Ethical legitimacy
        - Full explainability
        - Post-hoc evaluation

        **What This System Does NOT Do:**
        - No autonomous decisions
        - No ML optimization
        - No patient-level scoring
        - No real ABDM data
        - No sentiment analysis
        """)

        st.markdown("---")
```

```
        st.markdown("*Engineering Principle:*")
        st.info("If a decision cannot be explained in one sentence to a tired nurse at 2 a.m., it does not belong in this system.")

    # Initialize session state
    if 'simulation_running' not in st.session_state:
        st.session_state.simulation_running = False
    if 'simulation' not in st.session_state:
        st.session_state.simulation = None
    if 'report' not in st.session_state:
        st.session_state.report = None
    if 'completed' not in st.session_state:
        st.session_state.completed = False

    # Main content area
    if not st.session_state.simulation_running:
        # Welcome screen
        st.info("👈 Select your preferences and click 'Start Simulation' to begin")

        col1, col2 = st.columns(2)

        with col1:
            st.markdown("### 🎯 How This Works")
            st.markdown("""
            This is a deterministic operations orchestration simulator that models hospital queue management
            under resource scarcity while preserving safety, human authority, and ethical governance.

            Every simulation tick executes the immutable Agent Loop, which ensures consistent and explainable behavior.

            **The Agent Loop:**

            **PERCEIVE** → New patient arrivals are detected and logged

            **CLASSIFY** → Patients are triaged using two-stage classification

            **ORDER** → Queues are organized by urgency bands with FIFO ordering

            **CHECK** → Governance rules detect unsafe conditions

            **SURFACE** → Recommendations are presented to humans (never auto-executed)

            **LOG** → Every action is recorded for accountability
            """)

        with col2:
```

st.markdown("### 📊 Post-Run Evaluation")
st.markdown("""
After the simulation completes, three scores measure different aspects of system performance. These scores are computed after the simulation runs and never influence the live logic.

**IES - Institutional Efficacy Score**

This measures five pillars of institutional performance, including safety preservation, dignity and fairness, flow stability, capacity adaptation, and human authority integrity.

**PSS - Patient Satisfaction Score**

This measures fairness and predictability from the patient perspective, not happiness or wait time. It uses a weighted formula considering silent acceptance, explanation success, deferral trust, complaint penalty, and exit penalty.

**SSS - Staff Satisfaction Score**

This measures cognitive load and alert burden on staff, based on explanation load, de-escalation frequency, override clustering, and alert density.
""")

st.markdown("---")

st.markdown("### 🔑 Key Features")

feature_col1, feature_col2, feature_col3 = st.columns(3)

with feature_col1:
st.markdown("**🛡️ Safety First**")
st.markdown("Red-flagged patients are prioritized and never demoted through the irrevocable RED rule, ensuring critical cases always receive immediate attention.")

with feature_col2:
st.markdown("**👥 Human Authority**")
st.markdown("The system suggests and recommends actions but never executes them autonomously, preserving human decision-making authority at every step.")

with feature_col3:
st.markdown("**📝 Full Explainability**")
st.markdown("Every action is logged with complete details, allowing full traceability and accountability for all decisions and recommendations made during the simulation.")

else:
# Simulation is running
sim = st.session_state.simulation

```
# Progress indicators
progress_container = st.container()
with progress_container:
    col1, col2, col3, col4 = st.columns(4)

    with col1:
        progress_pct = (sim.current_time / sim.duration) * 100
        st.metric("⏱️ Simulation Progress", f"{sim.current_time}s / {sim.duration}s")
        st.progress(progress_pct / 100)

    with col2:
        st.metric("👥 Total Patients", len(sim.patients))

    with col3:
        st.metric("✅ Admitted", len(sim.admitted_patients))

    with col4:
        waiting_count = sum(len(q) for q in sim.queue.values())
        st.metric("⏳ Waiting", waiting_count)

st.markdown("---")

# Live view tabs
tab1, tab2, tab3, tab4 = st.tabs([
    "📊 Queue Status",
    "📋 Event Log",
    "💬 Social Layer",
    "🏥 Room Status"
])

with tab1:
    st.subheader("Current Queue Status")

    col1, col2 = st.columns([1, 2])

    with col1:
        st.markdown("**Queue Lengths by Urgency Band**")

        # RED queue
        red_count = len(sim.queue["RED"])
        st.markdown(f"🔴 **RED (Critical):** {red_count}")
        if red_count > 0:
            st.caption(f"Patients: {', '.join(str(p.id) for p in sim.queue['RED'][:5])}")

        # YELLOW queue
        yellow_count = len(sim.queue["YELLOW"])
        st.markdown(f"🟡 **YELLOW (Urgent):** {yellow_count}")
        if yellow_count > 0:
```

```
                st.caption(f"Patients: {', '.join(str(p.id) for p in sim.queue['YELLOW'][:5])}")

            # BLUE queue
            blue_count = len(sim.queue["BLUE"])
            st.markdown(f"🔵 **BLUE (Routine):** {blue_count}")
            if blue_count > 0:
                st.caption(f"Patients: {', '.join(str(p.id) for p in sim.queue['BLUE'][:5])}")

        with col2:
            st.markdown("**Recent Patient Arrivals**")

            recent_patients = sim.patients[-5:][::-1]  # Last 5, reversed

            if recent_patients:
                for patient in recent_patients:
                    triage_color = {
                        "RED": "🔴",
                        "YELLOW": "🟡",
                        "BLUE": "🔵",
                        "NOT_RED": "⚪"
                    }
                    stage_1 = triage_color.get(patient.triage_stage_1, "❓")
                    stage_2 = triage_color.get(patient.triage_stage_2, "❓") if
patient.triage_stage_2 else "⏳"

                    status_emoji = {
                        PatientStatus.WAITING: "⏳",
                        PatientStatus.ADMITTED: "✅",
                        PatientStatus.DEFERRED: "📅",
                        PatientStatus.TRANSFERRED: "🚑"
                    }
                    status_icon = status_emoji.get(patient.status, "❓")
                    st.markdown(
                        f"{status_icon} **Patient {patient.id}** | "
                        f"Age: {patient.age} | "
                        f"Triage: {stage_1}→{stage_2} | "
                        f"*{patient.chief_complaint[:40]}...*"
                    )
            else:
                st.info("No patients yet")

    with tab2:
        st.subheader("Event Log (Latest 15)")

        recent_events = sim.events[-15:][::-1]

        if recent_events:
            for event in recent_events:
```

```
                event_emoji = {
                    "PATIENT_ARRIVED": "🚶",
                    "TRIAGE_STAGE_1_ASSIGNED": "🏷️",
                    "TRIAGE_STAGE_2_ASSIGNED": "🔬",
                    "QUEUE_REORDERED": "🔄",
                    "ESCALATION_SUGGESTED": "⚠️",
                    "PATIENT_ADMITTED": "✅",
                    "PATIENT_DEFERRED": "📅",
                    "SEPARATION_UNSAFE": "🚨"
                }
                emoji = event_emoji.get(event.type, "📌")

                with st.expander(
                    f"{emoji} [{event.timestamp}s] {event.type.replace('_', ' ').title()}",
                    expanded=False
                ):
                    if event.entity_id:
                        st.markdown(f"**Patient ID:** {event.entity_id}")

                    if event.details:
                        st.json(event.details, expanded=False)
            else:
                st.info("No events logged yet")

        with tab3:
            st.subheader("Social Layer - Chat Bubbles")
            st.caption("Effects only, never causes. Reflects system state but does not influence logic.")

            recent_bubbles = sim.chat_bubbles[-10:][::-1]

            if recent_bubbles:
                for bubble in recent_bubbles:
                    actor_config = {
                        "Patient": ("👤", "human"),
                        "Staff": ("👨‍⚕️", "assistant"),
                        "Doctor": ("🩺", "assistant"),
                        "System": ("🖥️", "ai")
                    }
                    emoji, avatar = actor_config.get(bubble.actor, ("💬", "human"))

                    with st.chat_message(avatar):
                        st.markdown(f"{emoji} **{bubble.actor}** [{bubble.timestamp}s]")
                        st.markdown(f"*{bubble.message}*")
                        st.caption(f"Context: {bubble.context}")
            else:
                st.info("No chat activity yet")
```

```
        with tab4:
            st.subheader("Room Utilization")

            for room in sim.rooms:
                col1, col2 = st.columns([3, 1])

                with col1:
                    utilization = (room.current_load / room.capacity_per_minute * 100) if
room.capacity_per_minute > 0 else 0
                    st.progress(
                        utilization / 100,
                        text=f"**{room.name}** ({room.room_type}):
{room.current_load}/{room.capacity_per_minute}"
                    )

                with col2:
                    st.metric("", f"{utilization:.0f}%")

        # Run simulation ticks
        if sim.current_time < sim.duration and not st.session_state.completed:
            # Run several ticks for smoother updates
            for _ in range(3):
                if sim.current_time < sim.duration:
                    sim.tick()
            st.rerun()

        # Simulation complete - generate report
        elif sim.current_time >= sim.duration and st.session_state.report is None:
            with st.spinner("🔄 Generating post-run evaluation..."):
                st.session_state.report = sim.generate_report()
                st.session_state.completed = True
            st.rerun()

        # Display post-run report
        if st.session_state.report and st.session_state.completed:
            st.markdown("---")
            st.success("✅ Simulation Complete!")

            st.header("📈 Post-Run Evaluation Report")
            st.markdown("*All scores computed after simulation, never shown live, never feed
logic*")

            report = st.session_state.report

            # Summary statistics
            st.subheader("📊 Summary Statistics")

            col1, col2, col3, col4, col5 = st.columns(5)
```

```
        with col1:
            st.metric("Total Patients", report["summary"]["total_patients"])

        with col2:
            st.metric("✅ Admitted", report["summary"]["admitted"])

        with col3:
            st.metric("⏳ Still Waiting", report["summary"]["waiting"])

        with col4:
            st.metric("📋 Total Events", report["summary"]["total_events"])

        with col5:
            st.metric("⚠️ Escalations", report["summary"]["escalations"])

        st.markdown("---")

        col1, col2, col3 = st.columns(3)

        with col1:
            st.metric("🔴 RED Cases", report["summary"]["red_patients"])

        with col2:
            st.metric("🟡 YELLOW Cases", report["summary"]["yellow_patients"])

        with col3:
            st.metric("🔵 BLUE Cases", report["summary"]["blue_patients"])

        if report["summary"]["admitted"] > 0:
            st.metric(
                "⏱️ Avg Wait Time",
                f"{report['summary']['avg_wait_time']:.1f}s"
            )

        st.markdown("---")

        # IES Breakdown
        st.subheader("🏛️ Institutional Efficacy Score (IES)")
        st.markdown("""
        This score measures five pillars of institutional performance. Each pillar is scored
from 0 to 100,
        reflecting how well the system maintained safety, fairness, stability, adaptability, and
human authority
        during the simulation.
        """)

        ies = report["ies"]
```

```
        col1, col2 = st.columns([2, 1])

        with col1:
            for pillar, score in ies.items():
                if pillar != "Overall":
                    # Color code based on score
                    if score >= 80:
                        bar_color = "🟢"
                    elif score >= 60:
                        bar_color = "🟡"
                    else:
                        bar_color = "🔴"

                    st.progress(
                        score / 100,
                        text=f"{bar_color} **{pillar}**: {score}/100"
                    )

        with col2:
            overall_score = ies["Overall"]
            st.metric("Overall IES", f"{overall_score}/100")

            if overall_score >= 80:
                st.success("Excellent performance")
            elif overall_score >= 60:
                st.info("Good performance")
            elif overall_score >= 40:
                st.warning("Room for improvement")
            else:
                st.error("Needs attention")

        st.markdown("---")

        # PSS and SSS
        col1, col2 = st.columns(2)

        with col1:
            st.subheader("😊 Patient Satisfaction Score (PSS)")
            st.markdown("""
            This score measures fairness and predictability from the patient perspective. It
uses the exact formula
            from the specification, weighing factors like silent acceptance, explanation
success, deferral trust,
            complaint penalty, and exit penalty. Note that this measures fairness, not
happiness or wait time.
            """)
```

```
            pss = report["pss"]
            st.metric("PSS", f"{pss}/100")
            st.progress(pss / 100)

            if pss >= 75:
                st.success("Patients experienced fair and predictable care")
            elif pss >= 60:
                st.info("Moderate patient satisfaction with fairness")
            elif pss >= 40:
                st.warning("Patient experience needs attention")
            else:
                st.error("Significant fairness concerns")

        with col2:
            st.subheader("👨‍⚕️ Staff Satisfaction Score (SSS)")
            st.markdown("""
            This score measures the cognitive load and alert burden placed on staff during the simulation.
            It considers explanation load, alert density, de-escalation frequency, and override clustering.
            Higher scores indicate more manageable workload for staff.
            """)

            sss = report["sss"]
            st.metric("SSS", f"{sss}/100")
            st.progress(sss / 100)

            if sss >= 75:
                st.success("Manageable staff workload")
            elif sss >= 60:
                st.info("Moderate staff burden")
            elif sss >= 40:
                st.warning("High staff cognitive load")
            else:
                st.error("Excessive staff burden")

        st.markdown("---")

        # Legitimacy signals
        st.subheader("✅ Legitimacy Signals")

        st.markdown("""
        This simulation demonstrates the following key legitimacy principles that ensure ethical and safe operation. Every element of the system is designed to preserve human authority and maintain transparency in all decisions.
```

**Human Authority Preserved:** All recommendations from the governance system require explicit human approval. The system suggests actions but never executes them autonomously, ensuring that trained medical professionals make all final decisions.

**Safety First:** RED patients are prioritized through the irrevocable RED rule, meaning once a patient is classified as critical, they can never be demoted to a lower urgency level. This ensures critical cases always receive immediate attention.

**Full Explainability:** Every decision and action is logged with complete details, including timestamp, entity involved, and contextual information. This creates a complete audit trail for accountability and enables post-hoc review of all decisions made during the simulation.

**No Autonomous Action:** The system operates on a recommend-only basis. All escalations, room morphing suggestions, and capacity adjustments are surfaced to humans for decision-making rather than being executed automatically.

**Ethical Governance:** The separation monitor actively detects unsafe clustering of high-risk patients and triggers escalation pathways to prevent dangerous situations from developing. This protective mechanism operates independently of queue ordering and ensures patient safety is never compromised.
""")

st.markdown("---")

# Failure narrative
st.subheader("🔍 One Failure Narrative")

# Analyze the simulation for interesting challenges or failures
if report["summary"]["escalations"] > 0:
    escalation_events = [e for e in sim.events if e.type ==
"ESCALATION_SUGGESTED"]

    st.warning(f"""
    **Capacity Challenge Detected**

    During this simulation, the governance system detected
{report["summary"]["escalations"]} instances
    where intervention was recommended. This indicates periods of high demand that exceeded normal capacity.

    **What happened:** The separation monitor detected unsafe clustering of RED patients, meaning multiple
    critical cases arrived within a short time window. This triggered the governance gate to suggest
    external referral, room morphing, and reappointment options.

```
            **System response:** Rather than making autonomous decisions, the system
surfaced these recommendations
            to human decision-makers at timestamps: {', '.join(str(e.timestamp) for e in
escalation_events[:3])}s.

            **Outcome:** {report["summary"]["admitted"]} of
{report["summary"]["total_patients"]} patients were
            admitted, with {report["summary"]["waiting"]} still waiting at simulation end.
Average wait time was
            {report['summary']['avg_wait_time']:.1f} seconds.

            **Key insight:** This demonstrates the system's commitment to surfacing problems
rather than hiding
            them, and preserving human authority even during high-pressure situations.
            """)

        elif report["summary"]["waiting"] > 5:
            st.info(f"""
            **Queue Management Challenge**

            The simulation concluded with {report["summary"]["waiting"]} patients still in the
queue, indicating
            that demand slightly exceeded capacity during the simulation period.

            **What happened:** Patient arrivals and triage classifications resulted in a queue
that grew faster
            than the available rooms could process, even with proper urgency-based
prioritization.

            **System response:** The system maintained proper queue ordering with RED
patients first, then YELLOW, then BLUE,
            and logged all state changes. No patients were demoted from higher urgency
levels, preserving the
            irrevocable RED rule throughout the simulation.

            **Key insight:** This demonstrates realistic queue dynamics under resource
constraints, and shows how
            the system maintains safety and fairness even when capacity is insufficient to
serve all patients immediately.
            """)

        else:
            st.success(f"""
            **Smooth Operation**

            This simulation ran without major governance escalations. The patient arrival rate
and acuity mix
```

```
            remained within the hospital's capacity to manage safely throughout the entire
simulation period.

            **Outcome:** {report["summary"]["admitted"]} of
{report["summary"]["total_patients"]} patients were
            successfully admitted, with an average wait time of
{report['summary']['avg_wait_time']:.1f} seconds.

            **System behavior:** Even during smooth operations, the system continuously
monitored for safety risks,
            maintained full event logging for every action, and preserved all governance
checks. The absence of escalations indicates
            appropriate capacity matching between patient arrival patterns and available
resources.

            **Key insight:** This demonstrates that the system doesn't over-alert or create
unnecessary noise, only surfacing recommendations
            when genuinely needed. This measured approach helps prevent alert fatigue in
staff while maintaining vigilance.
            """)

        st.markdown("---")

        # Download options
        st.subheader("📥 Export Data")

        col1, col2 = st.columns(2)

        with col1:
            # Event log export
            event_log_json = json.dumps(
                [{"timestamp": e.timestamp, "type": e.type, "entity_id": e.entity_id, "details":
e.details}
                 for e in sim.events],
                indent=2
            )
            st.download_button(
                label="Download Event Log (JSON)",
                data=event_log_json,
                file_name="event_log.json",
                mime="application/json"
            )

        with col2:
            # Report export
            report_json = json.dumps(report, indent=2)
            st.download_button(
                label="Download Report (JSON)",
```

```
                data=report_json,
                file_name="simulation_report.json",
                mime="application/json"
            )

if __name__ == "__main__":
    main()
```

# VISUALIZATION DESIGN SPEC

# VISUALIZATION DESIGN SPEC

## “Living Hospital” Simulation UI (SimCity-lite, Governance-safe)

### Purpose of the Visualization (explicitly stated)

The visualization exists to:

- make **invisible operational stress visible**
- let non-technical stakeholders *feel* queueing, congestion, unfairness, and relief
- communicate **system behavior**, not UI polish

This is **deliberately cute** to lower defenses, while the logic underneath is strict.

---

# 1. CORE VISUAL METAPHOR

### 1.1 The Hospital as a Small Town

The hospital is represented as a **top-down miniature map**, like a SimCity neighborhood:

- Patients = **tiny circular tokens (“people”)**
- Rooms = **buildings**
- Queues = **footpaths / waiting lines**
- Movement = **teleporting hops between zones per time tick**

No smooth animation. Movement happens in *steps* to emphasize decisions, not physics.

---

# 2. PATIENT TOKENS (“PEOPLE”)

### 2.1 Visual Design

Each patient is a small icon with:

- **Color** = clinical urgency
    - 🔴 Red = emergency
    - 🟠 Orange = urgent
    - 🟡 Yellow = semi-urgent
    - 🟢 Green = routine
    - 🔵 Blue = preventive / deferrable
- **Shape modifier** (optional but powerful):
    - Circle = stable
    - Triangle = worsening
    - Diamond = waiting too long

This lets people *see* ethical tension.

### 2.2 Identity & Tooltips

Hovering over a patient token shows a tooltip:

```
Patient #42
Age: 58
Complaint: left arm pain
History: HTN, Dyslipidemia
Current score: 9.2 (RED)
Wait time: 3m 10s
```

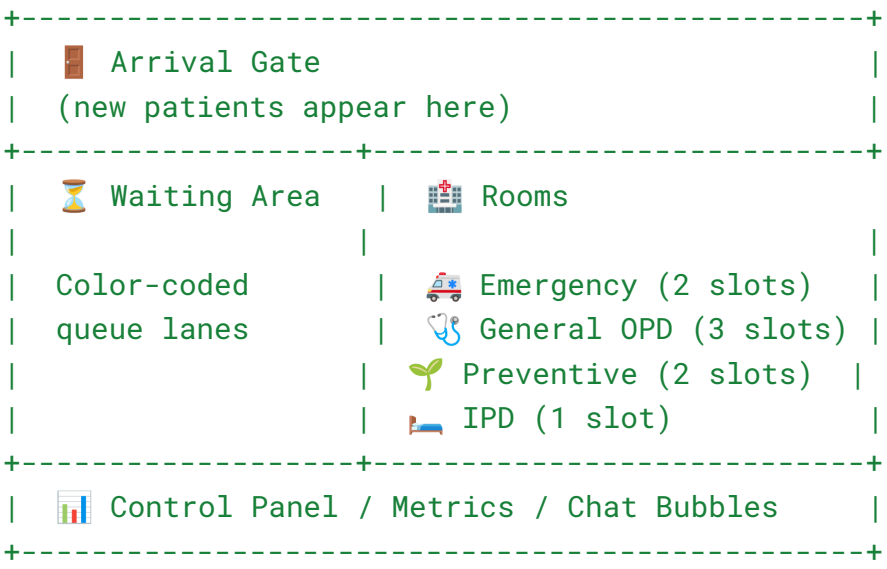
Important:
No names. No PHI. This reinforces governance.

---

# 3. SPATIAL LAYOUT (THE MAP)

### 3.1 Zones

The screen is divided into **five fixed zones**:




This layout never changes. Stability matters.

---

# 4. QUEUES AS VISIBLE OBJECTS

## 4.1 Queue Lanes

In the waiting area:

- Each urgency level has its **own visible lane**
- Patients stack vertically, newest at the bottom

This makes it instantly obvious when:

- Green cases pile up
- Reds jump the line
- Something is starving

## 4.2 Ethical Highlighting

When a patient is **reordered**:

- Their token briefly **glows**

- A small label appears:

  “Priority override: safety”

This is critical. It preempts complaints.

---

# 5. ROOMS AS BUILDINGS

## 5.1 Room Design

Each room is a **box with capacity slots**:

Example: Emergency Room

```
🚑 Emergency
[ • ][ • ][   ]
```

- Filled slot = occupied
- Empty slot = capacity
- Overload = red outline + warning icon ⚠️

## 5.2 Intake Rates (Tension Mechanism)

Each room has:

- a **service timer**
- visibly ticking down

Example:

“Next slot free in: 25s”

This creates *real tension* when queues grow.

---

# 6. TIME & MOTION MODEL

### 6.1 Time Scale

- One tick = **5 seconds**
- Patients arrive every tick
- Simulation runs in discrete steps

Controls:

- ▶️ Run
- ⏸️ Pause
- ⏭ Advance 30s
- 🔄 Reset

This mirrors:

- tabletop simulations
- hospital disaster drills
- ethics committee reviews

---

# 7. CHAT BUBBLES (THIS IS HUGE)

### 7.1 Patient Chat Bubbles

Patients occasionally emit chat bubbles above their token:

- “Why is he going before me?”
- “I’ve been waiting a long time…”
- “My pain is getting worse.”

These are triggered by:

- long wait times

- being deprioritized
- worsening condition

### 7.2 Staff Chat Bubbles

Rooms emit staff comments:

- “We’re overloaded.”
- “Need another room.”
- “This patient should’ve gone to emergency.”

These bubbles:

- humanize the system
- show cognitive load
- justify agentic actions

---

# 8. THE AGENT AS AN INVISIBLE OPERATOR

The AI agent is **not an avatar**.

It manifests only through:

- reordering
- rerouting
- opening/closing rooms
- triggering referrals
- suggesting reappointments

Whenever it acts, a small log appears:

> 🧠 Agent action: rerouted Patient #42 to Emergency
> Reason: symptom-history mismatch (cardiac risk)

This reinforces trust.

---

# 9. METRICS PANEL (BOTTOM BAR)

Live updating:

- Average wait time (by color)
- Safety breaches avoided
- Patient dissatisfaction score
- Staff stress score
- Ethics overrides count

These numbers *respond* to visual chaos.

---

# 10. WHY “CUTE” IS DELIBERATE (ENGINEER NOTE)

Include this comment in the codebase:

> “The visual style is intentionally simplified to reduce intimidation and invite exploration by clinicians, administrators, and policy stakeholders. Serious systems are best understood when they can be played with safely.”

This disarms critics.

---

# 11. WHAT NOT TO BUILD (IMPORTANT)

Explicitly tell the engineer **not** to:

- add smooth animations
- add scores floating everywhere
- gamify rewards
- show leaderboards

- add sound effects

This is not entertainment. It's **cognitive scaffolding**.

---

# 12. DEMO FLOW (FOR YOU)

When it is demoed it should:

1. Start paused
2. Let people observe empty hospital
3. Start simulation
4. Let queues build
5. Discomfort appears
6. Turn on agent
7. Let relief become visible
8. Show metrics change
9. Stop

# Visual System, Time Model, and Interaction Contract

**Project:** Living Hospital Orchestration Simulator
**Audience:** Software Engineer (Python / Streamlit)
**Mode:** Deterministic simulation with symbolic spatial visualization
**Default UI Mode:** Light (hospital-native)

---

# 1. Purpose of This Layer

This layer exists to **make system behavior legible**.

It is not a game engine.
It is not an animation project.
It is not a dashboard.

It is a **symbolic spatial representation** of an operational healthcare system under stress.

The visualization must:

- expose congestion
- surface ethical tension
- show trade-offs
- remain explainable under scrutiny by clinicians, administrators, and governance stakeholders

---

# 2. Architectural Separation (Non-Negotiable)

The system must be split into **three strictly separated layers**:

## 2.1 Simulation Core (already defined in Part 1)

- Owns all truth
- Owns time
- Owns patient state, queues, rooms, scores
- No UI logic

## 2.2 Visual State Adapter (this part)

- Reads simulation state
- Produces a **pure visual snapshot**
- No mutation
- No business logic

## 2.3 Renderer (Streamlit UI)

- Consumes visual snapshot
- Draws UI

- Handles user controls (pause, step, reset)

**UI must never directly access or mutate simulation internals.**

This mirrors real hospital systems:

- data layer
- interpretation layer
- display layer

---

# 3. Visual State Abstraction

At every simulation tick, generate a `visual_state` object.

## 3.1 Visual State Structure (Conceptual)

```
visual_state = {
    "time": current_time_seconds,
    "arrival_gate": [patient_ids],
    "queues": {
        "RED": [patient_ids],
        "ORANGE": [...],
        "YELLOW": [...],
        "GREEN": [...],
        "BLUE": [...]
    },
    "rooms": {
        "Emergency": {
            "capacity": 2,
```

```
        "occupied": [patient_ids],
        "time_to_next_free": seconds
    },
    "General_OPD": {...},
    "Preventive": {...},
    "IPD": {...}
},
"chat_bubbles": [
    {
        "source": "patient" | "staff",
        "patient_id": optional,
        "room": optional,
        "message": string,
        "severity": low | medium | high
    }
],
"metrics": {
    "avg_wait_by_color": {...},
    "patient_satisfaction": float,
    "staff_stress": float,
    "ethics_overrides": int
},
"recent_agent_actions": [
    {
```

```
            "action": string,
            "reason": string,
            "timestamp": time
        }
    ]
}
```

This structure is the **only contract** between logic and UI.

---

# 4. Visual Grammar (UI Semantics)

The visualization must follow a **fixed grammar**.

## 4.1 Patient Tokens

Each patient is rendered as a small circular token.

Attributes:

- **Color** → urgency level
    - Red, Orange, Yellow, Green, Blue
- **Outline / Glow** → ethical override or reprioritization
- **Shape modifier (optional)**:
    - Circle: stable
    - Diamond: waiting too long
    - Triangle: worsening

Hover tooltip shows:

- age

- chief complaint
- key history flags
- urgency score
- wait time

No names. No identifiers beyond numeric IDs.

---

### 4.2 Queues

Queues are vertical lanes in the waiting area.

Rules:

- One lane per urgency category
- Patients stack top-to-bottom
- New arrivals appear at the bottom
- Reordering causes a brief highlight

This allows immediate visual detection of:

- starvation
- priority inversion
- overload

---

### 4.3 Rooms

Rooms are boxes with explicit capacity slots.

Example representation:

```
🚑 Emergency
[ ● ][ ● ]
```

```
Next slot free in: 20s
```

Rules:

- Filled slot = occupied
- Empty slot = free
- Over-capacity = red outline + warning icon

Room intake rate must be **visible**, not inferred.

---

# 5. Time Model & Controls

## 5.1 Time Model

- Discrete ticks
- One tick = 5 seconds simulated time
- All state transitions occur only on ticks

No continuous animation.

## 5.2 Controls (Required)

- ▶️ Run
- ⏸️ Pause
- ⏭ Step (5s)
- ⏭⏭ Step (30s)
- 🔄 Reset

Simulation must be **reproducible**:

- same seed → same behavior

This is critical for credibility.

### 5.3 Time Control Interface (Required)

The simulator must expose explicit time controls:

- Play (continuous advancement)
- Pause (state frozen)
- Step (single tick, 5s)
- Speed selection (e.g. 0.5×, 1×, 2×)

These controls exist to support:

- inspection of agent decisions
- ethics review
- staff training
- post-event analysis

*“This system is designed to be examined, not just run.”*

---

## 6. Chat Bubbles (Human Friction Layer)

Chat bubbles are not decorative.
They are **measurable signals**.

### 6.1 Patient Bubbles

Triggered by:

- long wait
- reprioritization
- worsening state

Examples:

- “Why did they go before me?”
- “I’ve been waiting too long.”
- “My pain is getting worse.”

### 6.2 Staff Bubbles

Triggered by:

- overload
- ethical tension
- repeated overrides

Examples:

- “Emergency is full.”
- “This patient should be escalated.”
- “We need more capacity.”

These bubbles feed into:

- patient satisfaction score
- staff stress score

---

## 7. Agent Visibility

The agent must **never appear as a character**.

Its presence is shown only via:

- state changes
- routing decisions

- log entries

Example log entry:

```
Agent action:

Rerouted Patient #42 to Emergency

Reason: symptom-history mismatch (cardiac risk)
```

This reinforces:

- humility
- auditability
- governance alignment

---

# 8. Metrics Panel (Bottom Section)

Live metrics must update every tick:

- Average wait time by urgency
- Patient satisfaction score
- Staff stress score
- Ethics override count

Metrics should visibly worsen under chaos and improve under good orchestration.

---

# 9. UI Mode & Design Constraints

## 9.1 Light Mode (Default)

Justification:

- hospital-native
- fatigue-aware
- governance-safe
- avoids “toy” aesthetics

Design rules:

- high contrast
- neutral colors
- no gradients
- no neon
- no playful fonts

Dark mode may exist as a toggle, but is not primary.

---

## 10. Explicit Non-Goals (Do Not Build)

The following actively reduce seriousness and must not be implemented:

- smooth animations
- physics-based movement
- avatars
- sound effects
- leaderboards
- gamified rewards

If it looks like a game, it has failed.

---

## 11. Implementation Order (Recommended)

1. Implement visual_state adapter
2. Render queues and rooms statically
3. Add patient tokens
4. Add time controls
5. Add chat bubbles
6. Add metrics
7. Add agent action log

Stop as soon as system behavior is legible.

---

## 12. Engineering Principle

Include this comment in the codebase:

> “This visualization is intentionally simplified. Its purpose is to make operational stress, ethical trade-offs, and system behavior visible to humans who make real decisions.”

If the UI makes someone smile briefly and then go quiet, it is working.

# Metrics, Scoring, Evaluation Framework, and Institutional Profiles

**Project:** Living Hospital Orchestration Simulator
**Audience:** Software Engineer (Python / Streamlit)
**Purpose:** Make system performance measurable, comparable, and governable

---

## 1. Why This Layer Exists

This layer answers one question:

> “How do we know this system is helping, and for whom?”

Hospitals are not optimized for a single outcome.
They balance **safety, fairness, throughput, staff wellbeing, and trust**.

Part 3 introduces:

- explicit performance signals
- institutional tuning profiles
- longitudinal evaluation

This is what separates **engineering** from **administration-grade systems**.

---

# 2. Core Philosophy of Metrics (Read Carefully)

Metrics must:

- be interpretable by non-engineers
- reflect human experience
- degrade visibly under stress
- never imply clinical certainty

No “accuracy %”.
No “AI score”.
Everything ties back to **lived operational reality**.

---

# 3. Primary Metric Categories

## 3.1 Patient Safety Score (PSS)

Measures how well high-risk patients are protected.

Inputs:

- wait time vs allowable threshold by urgency
- number of near-miss escalations

- symptom–history mismatch caught by agent

Example calculation (conceptual):

```
Start at 100
-10 for every RED patient exceeding safe wait
-5 for every ORANGE exceeding threshold
+2 for every escalation caught early
Clamp between 0–100
```

Displayed as:

> “Patient Safety: 82 / 100 (2 near-misses avoided)”

---

### 3.2 Patient Experience Score (PES)

Measures perceived fairness and comfort.

Inputs:

- average wait time by urgency
- number of patient complaint chat bubbles
- number of reprioritizations affecting low-urgency patients

Key rule:

> First-come-first-served violation is *not* a penalty by default.
> Lack of explanation is.

Displayed as:

> “Patient Experience: 67 / 100 (queue dissatisfaction rising)”

---

### 3.3 Staff Stress Score (SSS)

Measures cognitive and operational load.

Inputs:

- room overload duration
- staff complaint bubbles
- frequency of manual overrides
- sustained intake pressure

Displayed as:

> “Staff Stress: High (Emergency under load for 90s)”

This is **not** a failure signal.
It is a warning signal.

---

## 3.4 Ethical Intervention Count (EIC)

Counts how often the system **had to override naïve fairness**.

Examples:

- priority jumps
- forced escalations
- external referrals under overload

Important:
High EIC is **not bad** by itself.
High EIC + low transparency **is**.

Displayed as:

> “Ethical Overrides: 5 (all safety-driven)”

---

## 3.5 System Throughput Index (STI)

Measures flow efficiency without fetishizing speed.

Inputs:

- patients processed per minute

- average room utilization
- backlog growth rate

Used mainly for **administrators**, not clinicians.

---

# 4. Composite Institutional Efficacy Score (IES)

This is the **headline score**.

It is a weighted composite of:

- Patient Safety
- Patient Experience
- Staff Stress (inverse)
- Throughput

## 4.1 Why a Composite Exists

Executives need a single number *with context*.
Doctors should never see this number alone.

Displayed as:

> “Institutional Efficacy: 74 / 100 (safety prioritized)”

---

# 5. Institutional Profiles (Critical Feature)

The same logic behaves differently under different **institutional constraints**.

The system must support **tunable profiles**.

## 5.1 Required Profiles

### A. Overburdened Government Hospital

- Safety weight: very high

- Experience weight: low
- Staff stress tolerance: high
- Room intake: slow
- External referrals: frequent

**B. Elite Private Hospital**

- Experience weight: high
- Staff stress tolerance: low
- Preventive routing favored
- Faster intake rates
- Lower tolerance for visible queues

**C. Balanced / Eka-Ideal Hospital**

- Safety first
- Transparent reprioritization
- Moderate intake rates
- Proactive room repurposing
- Reappointment and deferral used intelligently

## 5.2 Profile Implementation

Profiles are **parameter sets**, not logic forks.

Example:

```
PROFILE_GOVT = {
    "safety_weight": 0.5,
    "experience_weight": 0.2,
    "staff_weight": 0.1,
    "throughput_weight": 0.2,
    "max_wait_red": 0,
    "room_intake_modifier": 0.7
```

```
}
```

Switching profiles must:

- not change logic
- only change weights and thresholds

---

# 6. Longitudinal Evaluation (Not Just Snapshots)

Evaluation workflows assume access to time controls defined in Part 2.
Reviewers must be able to pause, step, and accelerate simulations in order to:

- inspect causality of agent decisions
- observe metric degradation under stress
- conduct ethics and safety reviews

The system must track:

- rolling averages
- degradation over time
- recovery after interventions

Graphs to include:

- wait time vs time
- safety score vs time
- staff stress vs time

This enables:

- after-action reviews
- ethics committee discussions
- ops planning

---

# 7. External Referral & Reappointment Logic (Governance-Aware)

## 7.1 External Referral Under Overload

When:

- emergency saturated
- wait exceeds safe threshold

Action:

- agent recommends referral
- logged explicitly as safety-driven
- reflected positively in safety score

## 7.2 Reappointment & Deferral

Low-urgency patients:

- offered reappointment
- moved out of active queue
- experience score improves if explained clearly

This mirrors real hospitals.

---

# 8. What NOT to Score

Explicitly do **not** score:

- diagnostic correctness
- clinical outcomes

- mortality
- treatment success

Those are outside scope and ethically unsafe.

---

# 9. Evaluation Output (For Demos)

At end of simulation, produce a summary:

- Which profile was used
- What trade-offs were made
- Who benefited
- Who waited longer
- Why that was acceptable

This is **gold** for exec conversations.

---

# 10. Final Principle (Put This in Code Comments)

> “These scores do not represent truth. They represent trade-offs made visible.”